Lecture 10 · Part II · Lecture notes

Regularising deep networks

Dataset: CIFAR-10

How to use these notes

The primary text for this lecture is Géron, Chapter 11. These notes are not a replacement for it and do not repeat it: they are the lecture's own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture's notebook, and the course checks that mechanically rather than trusting it.

Contents

1 The repaired network, trained for longer **2**

1.1 Name the gap . . . 2

2 Derivation: dropout as an average over subnetworks **3**

2.1 The expectation is preserved; the second moment is not . . . 3

2.2 Checked on the trained network . . . 4

3 Derivation: weight decay as per-step shrinkage **4**

3.1 Where AdamW puts it . . . 4

4 The cures, one at a time **5**

4.1 Early stopping . . . 5

4.2 Weight decay . . . 5

4.3 Dropout, and where it goes . . . 5

4.4 Monte Carlo dropout . . . 6

4.5 Data augmentation . . . 6

4.6 Label smoothing . . . 7

5 Alone, and on the ladder **7**

6 The failure condition nobody can see from the number **8**

7 What practitioners reach for **8**

8 Design exercise: the lesion check inside a dermatology app **9**

9 The notebook **10**

10 Exercises **10**

Summary **11**

---

1 The repaired network, trained for longer

Lecture 9 left us a twenty-layer network that trains: 3,072 inputs, twenty hidden layers of 100 units, each `Linear` → `BatchNorm` → `ReLU`, He initialisation, a linear head of ten outputs, cross-entropy, Adam at 0.001, batches of 128, gradient clipping at 1.0. Its ablation ladder ended on a rung that made things worse — dropout — and that rung is explained in these notes.

Train the same network on the same 10,000 images, with the same seed, for 30 epochs instead of 20, and log one more column: the accuracy on the *training* images after every epoch, measured in evaluation mode on all 10,000 of them.

Epoch	Training accuracy	Validation accuracy
20 — where Lecture 9 stopped	67.71%	40.26%
21 — the best validation epoch	68.90%	40.90%
30 — the last	78.05%	38.90%

Nine more epochs bought ten points on the training images and lost two on the validation images: a gap of 39.15 points at the end. The validation *loss* turned earlier than the validation accuracy — lowest at epoch 11, at 1.7145, and 2.0959 by the end — because the network's wrong answers became confident wrong answers. The test accuracy of the last weights is 40.11%; of the epoch-21 weights, 41.46%.

1.1 Name the gap

The error on new images that comes from fitting the *particular* sample — which pixels this frog has, rather than what a frog looks like — is variance. The arithmetic of why the network can afford it: 504,210 free parameters against 10,000 training images, 50.4 parameters per image, and the labels carry $10{,}000 \cdot \log_2 10 \approx 33{,}219$ bits of information. Half a million weights can store thirty thousand bits many times over, and given enough epochs they will.

Every cure is a constraint, and each trades a little bias — the constraint may be wrong — for less variance.

Cure	What it constrains	The knob
Early stopping	how far the fit runs	the epoch, chosen on validation
Weight decay	the size of the weights	λ
Dropout	which units may rely on which	p, and which layers
Data augmentation	the function: invariant to the copies	the transforms
Label smoothing	the target: no infinite logit	ε

2 Derivation: dropout as an average over subnetworks

At every training step, dropout sets each hidden unit to zero with probability p, independently. A mask $\mathbf{m} \in \{0,1\}^n$ chooses which of a layer's n units survive; each choice is a smaller, *thinned* network, and all 2^n of them share the same weights. One step samples one mask, runs that thinned network forward and backward, and updates the shared weights. With $n = 100$ there are $2^{100} \approx 10^{30.1}$ thinned networks of one layer alone — more than training steps by twenty-seven orders of magnitude, so almost all of them are trained zero times. What makes this work is the shared weights, and the question is what the full network, run without masks, has to do with their average.

The expectation under the mask

One unit of the next layer sees $z = \sum_j w_j a_j$ from activations a_j. Under a mask with $m_j \sim \text{Bernoulli}(1-p)$, independent across j, with the a_j and w_j held fixed,

$$\tilde{z} = \sum_j m_j\, w_j\, a_j, \qquad \mathbb{E}[\tilde{z}] = \sum_j \mathbb{E}[m_j]\, w_j a_j = (1-p)\, z$$

by linearity of expectation. Train with masks and run without them, and every pre-activation is $1/(1-p)$ times larger than anything the weights were trained on. **Inverted dropout** divides the kept activations by $1-p$ during training:

$$\tilde{z} = \frac{1}{1-p} \sum_j m_j\, w_j\, a_j \qquad \Longrightarrow \qquad \mathbb{E}[\tilde{z}] = z,$$

so the full network at test time — no masks, nothing rescaled — computes the expectation over masks. This is what `nn.Dropout` does.

2.1 The expectation is preserved; the second moment is not

For one term, $m_j^2 = m_j$ because $m_j \in \{0,1\}$, so

$$\mathbb{E}\left[\left(\frac{m_j a_j}{1-p}\right)^2\right] = \frac{\mathbb{E}[m_j]\, a_j^2}{(1-p)^2} = \frac{a_j^2}{1-p}.$$

Every dropout layer multiplies the second moment of the signal passing through it by $1/(1-p)$. The expectation is right; the spread around it is not, and on a deep stack the factor compounds

exactly as Lecture 9's per-layer factor ρ did: on one layer at $p = 0.5$ it is 2; on all twenty layers at $p = 0.1$ it is $(1-p)^{-20} = 8.23$; at $p = 0.5$ it is $2^{20} = 1{,}048{,}576$. A factor applied twenty times is a different thing from a factor applied once, which is why dropout goes on the top layer or two.

2.2 Checked on the trained network

The notebook takes the last hidden layer's activations for one batch and the head's weights from the trained network, draws 20,000 masks at $p = 0.5$, and averages the rescaled pre-activations: the average is within 0.0433 of z on pre-activations of mean size 1.999, the error falling roughly as one over the square root of the number of masks, and the measured second moment is 2.000 times the unmasked one. Across p from 0.1 to 0.6 the measured ratio lies on $1/(1-p)$.

What the result buys you

The network run without masks computes the expectation over 2^n thinned networks — an average of models that share weights, Lecture 5's cure for variance without the forest's bill. No unit can rely on a particular neighbour, because the neighbour is absent half the time. And keeping the masks on at test time gives the ensemble's spread as well as its mean: Monte Carlo dropout, a measure of how sure the network is, for the price of T forward passes.

3 Derivation: weight decay as per-step shrinkage

The step, written out

Add $\frac{\lambda}{2}\|\theta\|^2$ to the loss. Its gradient is $\lambda\theta$, so one gradient step is

$$\theta \;\leftarrow\; \theta - \eta\,(\nabla L + \lambda\theta) \;=\; (1-\eta\lambda)\,\theta \;-\; \eta\nabla L\,:$$

first shrink every weight by the factor $1-\eta\lambda$, then take the usual step. The ℓ_2 penalty and the per-step shrinkage are the same thing — for gradient descent.

With $\eta = 0.001$ and $\lambda = 0.2$ the factor is 0.9998 per step; over the 2,370 steps of a thirty-epoch run a weight that receives no gradient shrinks to 0.62 of itself. A weight that earns its size keeps it; one that does not, decays.

3.1 Where AdamW puts it

Adam divides each coordinate's step by a running estimate of its gradient's size. Put $\lambda\theta$ *into* the gradient and it is divided too: a weight with a large gradient history is barely decayed, one with a small history is decayed hard, and in the limit of no loss gradient at all every weight moves by about η towards zero whatever its size. **AdamW** applies the shrinkage $(1-\eta\lambda)\,\theta$ outside the adaptive step, as the formula says.

The notebook isolates the two with one optimiser step on a zero loss gradient, $\eta = 0.001$, $\lambda = 0.2$, on four weights of sizes 0.5, 0.05, 0.005 and 0.0005. SGD and AdamW multiply every one of them by 0.9998. Adam with the coupled penalty moves 0.5 to 0.4990 and 0.0005 to −0.0005:

the smallest weight crosses zero in a single step.

4 The cures, one at a time

4.1 Early stopping

The harness keeps a copy of the weights at the epoch with the best validation accuracy; the cure is to ship those rather than the last. Two rules: the epoch is chosen on the validation set, and what is reported is the test accuracy of the *kept* weights — not the validation accuracy that chose them, which is the maximum of thirty readings and optimistic by construction.

Measured: the last weights score 40.11% on the test set and the epoch-21 weights 41.46%, a gain of 1.35 points and nine epochs of training not spent memorising. It did not close the gap — at epoch 21 the training accuracy was already 68.90%. Early stopping stops the damage; it does not constrain the model.

4.2 Weight decay

AdamW with $\lambda = 0.2$, everything else unchanged.

	$\|\|\theta\|\|$ after epoch 30	Training	Validation	Gap	Test
no decay	83.9	78.05%	38.90%	39.15	40.11%
AdamW, $\lambda = 0.2$	54.6	75.19%	40.80%	34.39	41.34%

The weights are a third smaller, the gap is five points narrower, and the test accuracy is a point higher — a point being about the seed-to-seed spread, as the ladder shows. Without decay the norm *grew* over the run, from 77.7 after the first epoch to 83.9.

Failure condition — the penalty inside Adam

`Adam(weight_decay=0.2)` on this network: test accuracy 10.00%, chance, and $\|\|\theta\|\| = 0.2$ after thirty epochs against 83.9 without decay. The normalised penalty moves every weight by about η towards zero at every step whatever its size, and five hundred thousand weights of size 0.1 reach zero in a hundred steps. Nothing raises; the loss sits at ln 10 and the run completes. The same λ in `AdamW` is the second row of the table above. A value of λ that is right for one optimiser is wrong by orders of magnitude for the other.

4.3 Dropout, and where it goes

Dropout at 0.5 after the top hidden layer — the configuration the derivation points at — beside Lecture 9's last rung, dropout at 0.1 after every one of the twenty layers, both trained for the same thirty epochs.

	Train, ep. 20	Val, ep. 20	Train, ep. 30	Val, ep. 30	Gap	Test
none	67.71%	40.26%	78.05%	38.90%	39.15	40.11%
0.5 on the top layer	—	—	74.62%	41.24%	33.38	41.56%
0.1 on every layer	39.09%	33.90%	47.39%	37.08%	10.31	37.55%

On the top layer: a narrower gap and a point and a half on the test set, a modest cure on a network this narrow. On every layer: the narrowest gap in the table, bought by not learning.

Lecture 9's last rung, explained

That ladder stopped at epoch 20 and read dropout 0.1 on every layer as a loss of seven points. At epoch 20 the run stood at 33.90% on validation against 40.26% without dropout — the row Lecture 9 printed — and its *training* accuracy was 39.09%: it was not overfitting, it was barely fitting. The second moment of the signal was multiplied by 8.23 on its way up twenty layers of masks, and the network spent its epochs learning through that noise. Dropout on every layer of a deep, narrow stack is not a regulariser; it is a slower learner with a narrower gap. Lecture 9 read the rung as a regulariser added to a network that was not overfitting; the training column, which that deck did not log, says it was overfitting already.

4.4 Monte Carlo dropout

Keep the masks on at test time, run the network $T = 20$ times, and average the softmax outputs. Only the `Dropout` modules go back to training mode; batch normalisation stays in evaluation mode, or each pass normalises by a different batch and the spread measures the batches rather than the masks.

Masks off, the dropout network scores 41.81% on the test set; twenty passes averaged, 41.69% — no better. But the largest averaged probability is a confidence that means something: sorted into deciles by it, the most confident tenth of the test images is right 79.6% of the time and the least confident tenth 20.2%. Answer the 80% most confident and abstain on the rest, and the accuracy on what is answered is 46.81%. A confidence can be *acted on*.

4.5 Data augmentation

A photograph flipped left-to-right, or shifted by two pixels, is a photograph of the same thing. Training on a fresh random copy of every batch tells the network so: the function it fits has to give the same answer on an image and its copies, which rules out the part of the hypothesis space that was memorising pixel positions. Which transforms is a statement about the data: a flipped digit is a different digit, a flipped frog is a frog.

	Training	Validation	Gap	Test	Same answer on an image and its flip
none	78.05%	38.90%	39.15	40.11%	52.91%
flips and shifts	53.26%	44.58%	8.68	44.88%	71.77%

A gain of 4.77 points on the test set, and the gap cut from thirty-nine points to nine: the one cure that did more than narrow the gap. The last column is the constraint, measured. Without

augmentation the network changes its answer on half the test images when they are flipped — it had learned where the pixels were. With it, the flip is nearly invisible.

Failure condition — the evaluator's transform

Score the augmented network on a random copy of each test image instead of the image: 43.64% instead of 45.15%. The network did not change; the task did. Augmentation lives in the training path and only there; validation and test images are never transformed. The usual way to get this wrong is a single `transform` object shared by both loaders, and the usual symptom is nothing: a plausible number, a little low. The network trained without augmentation, on the same copies, scores 37.48%; both numbers are honest answers to a question nobody asked.

4.6 Label smoothing

Replace the one-hot target with $(1 - \varepsilon)$ on the true class and $\varepsilon/10$ on each of the other nine: with $\varepsilon = 0.1$, 0.91 and 0.01. The network is no longer rewarded for driving one logit to infinity, which was the mechanism that kept the training loss falling after the validation loss had turned. The training loss at epoch 30 is *higher* by construction, 1.1662 against 0.8052, because the target can no longer be reached; the test accuracy is 41.18% against 40.11%, and the gap barely moves, 36.40 against 39.15.

5 Alone, and on the ladder

Cure, alone, one seed	Training	Validation	Gap	Test
none: the network as it stands	78.05%	38.90%	39.15	40.11%
early stopping, the best epoch kept	78.05%	38.90%	39.15	41.46%
weight decay, AdamW $\lambda = 0.2$	75.19%	40.80%	34.39	41.34%
dropout 0.5 on the top layer	74.62%	41.24%	33.38	41.56%
dropout 0.1 on every layer	47.39%	37.08%	10.31	37.55%
augmentation: flips and shifts	53.26%	44.58%	8.68	44.88%
label smoothing 0.1	76.32%	39.92%	36.40	41.18%

Every cure narrows the gap. One of them raises the test accuracy by more than a point. Narrowing the gap and generalising better are not the same thing: a network that learns nothing has no gap at all.

The ladder continues Lecture 9's from where it ended, one cure per rung, each rung over three seeds. From the second rung on every rung is read at its best validation epoch, because early stopping is on the ladder and stays on.

Configuration	Test accuracy	sd over 3 seeds	Change	Gap
the network as it stands, last epoch	40.58%	±0.47	—	38.8
+ early stopping	41.39%	±0.58	+0.81	38.8
+ weight decay, $\lambda = 0.2$	41.59%	±0.67	+0.20	34.8
+ dropout 0.5, top layer	41.18%	±0.92	−0.41	30.1
+ augmentation	44.30%	±0.18	+3.12	6.5
+ label smoothing 0.1	44.55%	±0.90	+0.25	6.7

The typical spread, the mean of the sd column, is 0.62 points. One rung clears twice that: augmentation, +3.12. The last rung against the first is +3.97, and the other four rungs together are worth +0.85, inside the noise. The gap went from 38.8 points to 6.7.

The answer to "should I add dropout, or weight decay, or stop early?"

Yes, cheaply, and do not expect much. Every cure closed the gap; one of them raised the score, and it was the only constraint on the *function* rather than on the weights. The question with a large answer is "what does the network not need to learn?", and augmentation is how you tell it. Lecture 12 builds that invariance into the architecture instead, and the gain is larger again.

6 The failure condition nobody can see from the number

Six values of λ, one seed each, every run read at its best validation epoch.

λ	0	0.05	0.1	0.2	0.5	1.0
validation, best epoch	40.90%	41.32%	41.30%	41.54%	41.68%	41.46%
test, same weights	41.46%	41.86%	41.88%	41.71%	42.43%	41.76%

Failure condition — a knob tuned on the test set

Choose λ by the test row and report that number, and the reported number is the largest of six draws from the test set: optimistic by an amount that *cannot be seen from the number*. Here the two choices happen to coincide, at $\lambda = 0.5$, so the optimism was zero — which is known only because the validation row exists. With a thousand configurations it is the whole result. The validation set chooses; the test set is read once.

7 What practitioners reach for

Always: early stopping on a validation metric with the best weights kept, and augmentation that matches the data's invariances, in the training path only. Usually: AdamW with a small decay, and label smoothing at 0.1. Where it fits: dropout at 0.2–0.5 on wide, top layers and little on narrow or convolutional ones, and Monte Carlo dropout wherever a confidence will be acted on. Before all of them: more data, since the ratio of parameters to images is the cheapest knob of all; and then the architecture, because a convolution *is* the shift invariance built in rather than taught, and a pretrained backbone is a thousand hours of someone else's data.

8 Design exercise: the lesion check inside a dermatology app

The lecture closes with a design exercise in the manner of an ML system-design interview, on a system built with today's method.

The task. A consumer health company sells a skin-care app. Its new feature lets a user photograph a mole or lesion and get one of two answers: *looks fine, check again in a few months*, or *see a dermatologist — here is how to book one*. The model behind it is a classifier trained on a few thousand photographs labelled by dermatologists, far fewer than it has parameters. The business measures referrals that turn out to be warranted, users retained, and the cost of each visit it triggers. The actors are the product and data teams, the users, the dermatologists who label and who receive the referrals, and the medical-device regulator, because software that tells a person to see or not see a doctor is a medical device.

1. **Frame it.** One row per photograph; the label is the dermatologist's verdict, *refer* or *do not refer*; the output is a probability, and a threshold on it is the product decision. The two errors are not symmetric: a missed melanoma is a harm the company answers for, an unneeded visit is a cost. Fix the sensitivity first, as a clinical requirement agreed with the dermatologists and written into the regulatory file, and optimise everything else under it. *Pitfall:* accuracy — most lesions are benign, so a model that refers nobody is accurate and lethal.

2. **The label.** Some photographs carry a biopsy result, most carry one dermatologist's visual verdict: keep the source as a column, evaluate on the biopsy-confirmed rows, smooth the labels of the rest. Double-label a slice and treat the agreement between dermatologists as the ceiling any model can reach. *Pitfall:* clinic photographs are taken with a dermatoscope under ring light and the user's with a phone in a bathroom — collect a labelled phone set before shipping, or the training set is of a different task.

3. **Split by patient.** Every photograph of one patient goes to the same side, stratified by label and by clinic; validation chooses every knob, the test side is read once. *Pitfall:* a random split by photograph puts the same lesion, photographed twice, on both sides; the network memorises the patient's skin and the clinic's lighting, the validation score is a memory test it passes, and the number in the submission is discovered to be wrong in the field, on users.

4. **Augmentation that respects the lesion.** Flips, rotations by any angle, small crops and small changes of brightness, in the training path only. *Pitfall:* colour and scale are the signal — a hue shift turns a benign brown lesion into a suspicious black one, an aggressive zoom changes the diameter the clinical rule is built on. Choose the transforms with a dermatologist in the room and test each on the validation side.

5. **The model, and the cures on it.** A pretrained convolutional backbone, fine-tuned, with a dropout layer before the head; AdamW with a small decay; early stopping on the validation sensitivity at the agreed specificity, the best weights kept; label smoothing on the visually-labelled rows. *Pitfall:* a training curve that reaches perfect accuracy on the clinic photographs within a few epochs, and a team that watches only it and ships the last epoch. Log the validation curve from the first run, and keep the ablation table, because the regulator's reviewer will ask why each component is there.

6. **When the model is not sure, it says so.** Monte Carlo dropout at inference: the mean as the probability, the spread as the confidence; a third answer — *this photograph cannot be*

assessed, retake it or see a dermatologist — for the least confident fraction; and a cheap separate check that the photograph is of skin, in focus and lit. *Pitfall:* a two-answer product forces a confident verdict on a kind of image the model has never seen — a skin tone absent from the clinic data, a tattoo, a bandage. The least confident decile is where accuracy is lowest and where the users who most need a doctor are; abstain and refer.

7. **The threshold, the calibration, and who signs it.** The sensitivity target fixes the threshold on the validation side; calibrate on a held-out slice and check per skin tone, per clinic and per phone model; the threshold is a clinical decision signed by the medical director, not a hyperparameter. *Pitfall:* the team tries a dozen backbones and thresholds and reports the best test sensitivity — the largest of a dozen draws — and the regulator's own evaluation on fresh patients comes in lower. One test set, read once, by someone who did not choose the model.

8. **Release, monitor, retrain.** A limited cohort first, every referral followed up and every *looks fine* sampled and reviewed by a dermatologist; monitor the input distribution and the referral rate against the plan; retrain on the new phone-taken, reviewed photographs through the same split-by-patient pipeline and re-evaluate on the untouched test set before every release, because a changed model is a changed device. *Pitfall:* the feature changes who photographs what, and the labels that come back are from the referrals the model chose, so retraining on them teaches the model its own habits. The sampled review of the *looks fine* answers is the only unbiased signal the product produces.

9. **Ship it.** Clinic photographs with their label source → split by patient → augmentation agreed with a dermatologist → pretrained backbone, dropout, AdamW, early stopping → Monte Carlo dropout and a third answer → threshold signed off, test set read once → audited release → monitor, retrain, re-evaluate — each step named for the pitfall it avoids.

9 The notebook

What to change

Set `top` to 20 with dropout 0.5 in Section 11 and watch the validation curve stay at chance for all thirty epochs: 2^{20} is about a million, and that is the factor on the second moment of the signal by the time it reaches the head. Then add 2.0 and 5.0 to the sweep's grid in Section 17: the validation choice does not move, and the optimism cannot fall, because adding options can only raise a maximum.

10 Exercises

The five set at the end of this lecture's deck, in the style of the written paper. The solutions are on Lecture 11's deck and in the exercise book.

1. A hidden unit's activations a_j feed a pre-activation $z = \sum_j w_j a_j$. Dropout keeps each unit with probability $1 - p$. Derive $\mathbb{E}[\tilde{z}]$ under the mask, and state what inverted dropout divides by so that the network run without masks computes that expectation. [4]

2. Dropout preserves the expected pre-activation. Show that it does not preserve the second moment, give the factor per dropout layer, and say what that implies for a stack of twenty layers. [5]

3. Add $\frac{\lambda}{2}\|\theta\|^2$ to the loss and write one gradient-descent step. Then say what happens to that term when it is handed to Adam as part of the gradient, and how AdamW differs. [4]

4. A colleague trains for fifty epochs, keeps the weights of the epoch with the highest validation accuracy, and reports that validation accuracy as the model's result. Name the two things that are right about this and the one that is wrong. [4]

5. Six values of a weight-decay knob are tried, each run read at its best validation epoch. The value with the highest validation accuracy is reported on the test set, giving t_v; the value with the highest test accuracy is reported, giving t_t. State the sign of $t_t - t_v$, when it can be zero, and why the size of the bias cannot be read from either number. [4]

Summary

Lecture 9's repaired network, trained for thirty epochs, memorises: 78.05% on its training images against 38.90% on validation, a gap of thirty-nine points, with 504,210 parameters for 10,000 images. The gap is variance and every cure is a constraint.

Dropout trains one of 2^n thinned networks per step; inverted dropout's division by $1 - p$ makes the full network at test time the expectation over masks, checked on the trained network to within 0.0433 on pre-activations of size 1.999. The second moment grows by $1/(1 - p)$ per dropout layer, which is why one layer of it works and twenty do not, and why Lecture 9's last rung failed. Weight decay is the shrinkage $(1 - \eta\lambda)\,\theta$ before every step; handed to Adam as a penalty it is normalised away and the network sits at chance, while AdamW applies it outside the step.

Five cures alone and on a three-seed ladder: every one narrowed the gap, and augmentation alone raised the test accuracy beyond the seed spread, +3.12 points, because it constrains the function rather than the weights. Monte Carlo dropout turned confidence into a decision: answer the surest 80% at 46.81%, refer the rest. And a knob chosen on the test set reports a number whose optimism cannot be seen from the number: the validation set chooses, and the test set is read once.